Validation

Validation · Signup form accumulates all field errors · Build with Waqar

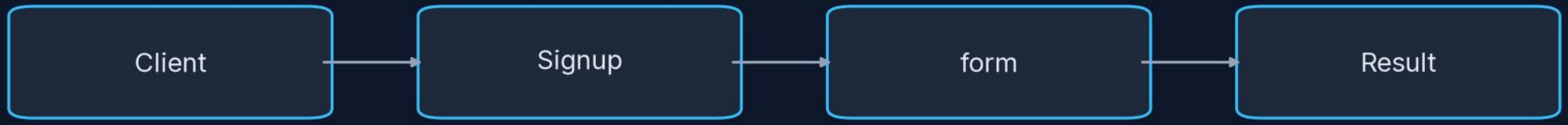
Today's Hook

- Problem: Signup form accumulates all field errors
- Pattern: Validation — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Signup form accumulates all field errors
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Validation: Signup form accumulates all field errors

Real-World Scenario

- Running example: Signup form accumulates all field errors
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.validation;

import com.buildwithwaqar.patterns.functional.validation.SignupValidation;
/** Problem: signup form must show EVERY field error at once. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Validation problem: signup accumulates errors ===");
    System.out.println(SignupValidation.message(SignupValidation.signup("a@b.c", 30)));
    System.out.println(SignupValidation.message(SignupValidation.signup("bad", 10)));
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.validation;

import dev.zazr.control.Validation;

record Signup(String email, int age) {}

public final class SignupValidation {
    private SignupValidation() {}

    public static Validation<String, String> email(String raw) {
        return Validation.fromPredicate(raw, e -> e != null && e.contains("@"), e -> "email has no @");
    }

    public static Validation<String, Integer> age(int raw) {
        return Validation.fromPredicate(raw, a -> a >= 18, a -> "age under 18");
    }

    public static Validation<String, Signup> signup(String email, int age) {
        return Validation.zipWith(email(email), age(age), Signup::new);
    }

    public static String message(Validation<String, Signup> v) {
        return switch (v) {
            case Validation.Valid(var s) -> "welcome " + s.email();
            case Validation.Invalid(var errors) -> errors.mkString(", ");
        };
    }
```

Solution (2)

```
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.validation;

import com.buildwithwaqar.patterns.functional.validation.SignupValidation;
/** Problem: signup form must show EVERY field error at once. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Validation problem: signup accumulates errors ===");
    System.out.println(SignupValidation.message(SignupValidation.signup("a@b.c", 30)));
    System.out.println(SignupValidation.message(SignupValidation.signup("bad", 10)));
  }
}
```

Walkthrough Output

```
=== Validation problem: signup accumulates errors ===
welcome a@b.c
email has no @, age under 18
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Validation.

Related Patterns

- Contrast Validation with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Validation → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.